Class06

Niam: A17805610

Table of contents

Background

All functions in R have at least of 3 things:

- a *name*
- input *arguments*
- the *body* (lines of R code that do the work of the function).

A first function

Here we will create a function to add some numbers. Let's call it `add()`.

Input arguments can be either "required" or "optional". The later will have a default value that will be used if the user does not specify them.

> Q1a. Your first version of the function should add two input numbers together. For example, add(4, 7) should return 11. [1 pt]

```
add <- function (x,y=0) {
  x + y
}
```

can we use our new function

```
add(10,100)
```

```
[1] 110
```

```
add (10)
```

```
[1] 10
```

Q1b. b. For you second version, adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add(c(4,7,10)). [1 pt]

```
add <- function (x,y=0) {
  sum(x,y)
}
```

```
add(4,7) #should return 11
```

```
[1] 11
```

```
add (c(4,7,10)) #should return 21
```

```
[1] 21
```

Q1c.Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) should return 2. Hint: explore the … (dots) argument or the base R function sum()

```
add <- function(...){
  return(sum(...))
}
  add(1,2,3,-4)
```

```
[1] 2
```

Notes from class:

```
add <- function (x,y=0, z=0) {
  sum(x,y,z)
}
```

```
add(10,100)
```

```
[1] 110
```

```
ans <- add(1,2,3)
ans
```

```
[1] 6
```

we can explicitylu set a **return** value output from a function (rather than the default last line) by using the 'return()'function call.

```
add <-  function (x, y=0, z=0) {
  return(sum(x,y,z))
  cat("Is it break time yet?")
}
```

Write a generate_dna() function

A useful function here is the "base R" 'sample() function:

```
sample(1:5, size=3)
```

```
[1] 5 2 4
```

```
sample(1:5, size=60, replace= TRUE)
```

```
 [1] 3 2 2 1 1 3 2 4 3 2 1 1 3 5 1 2 5 3 5 2 3 5 3 3 4 4 2 4 5 4 4 1 2 3 1 2 5 2
[39] 2 4 3 4 2 4 3 4 5 1 5 5 4 2 2 5 2 1 4 2 2 5
```

```
sample(x=c("A", "C", "G", "T"), size=10, replace= TRUE)
```

```
 [1] "C" "T" "G" "T" "C" "T" "A" "T" "C" "T"
```

We can use this to make a random nucletide sequence if we draw from :“A”, “C”, “G”, and “T”..

> Q2a: Write a function generate_dna() that returns a random DNA sequence of a length specified by the user. a. Your first version should return a multi-element vector of single character nucleotid

```
generate_dna <-function(len=100){
  sample(x=c("A", "C", "G", "T"), size=len, replace= TRUE)
}
```

```
generate_dna(len=100)
```

```
  [1] "T" "A" "A" "G" "A" "A" "C" "A" "A" "G" "G" "G" "G" "C" "T" "G" "C" "G"
 [19] "C" "A" "T" "A" "C" "C" "G" "A" "G" "C" "T" "G" "C" "T" "T" "G" "C" "C"
 [37] "T" "G" "G" "C" "T" "C" "A" "A" "C" "A" "T" "A" "C" "A" "C" "G" "A" "G"
 [55] "G" "G" "T" "T" "G" "T" "C" "A" "C" "C" "T" "G" "A" "C" "T" "A" "T" "A"
 [73] "C" "G" "G" "G" "T" "T" "T" "T" "T" "A" "T" "T" "A" "A" "G" "A" "T" "A"
 [91] "C" "T" "C" "A" "A" "A" "G" "T" "C" "G"
```

> Q2b Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a single character string (not a vector of single letters but a singe vector of multiple letters). For example “AAGGCTG”. [1 pt]

```
generate_dna <- function(len, single.element=TRUE) {
  #len provides the length of the DNA sequence
  #single.element: when true then it indicates to return as one string, if false then return
  ans <- sample(x=c("A", "C", "G", "T"), size=len, replace = TRUE)
  #cat("Hello...")

  if(single.element) {
    #cat("is it me you are looking for...")
   ans <-  paste( ans, collapse = "")
  }
#This is to return a single string with no space between character that is what collapse =""

  ## Format as FASTA with an ID line
  ##
  ##

  return(ans)
}
```

```
generate_dna(len=15)
```

```
[1] "TGTGCCTGTTGTTTA"
```

functions that are helpful here are `paste()`, `if()`, `cat()`, and `return()`

```
generate_dna(len=20, single.element = FALSE)
```

```
 [1] "A" "G" "A" "G" "T" "A" "G" "C" "A" "T" "A" "A" "T" "T" "G" "A" "C" "T" "C"
[20] "T"
```

Q2c.c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example:

```
>len9
CGAAGGCTG
```

```
cat("hello \n there")
```

```
hello
 there
```

```
generate_dna <- function(len=10, single.element=TRUE) {

  #generates nucleotides
  ans <- sample(x=c("A", "C", "G", "T"), size=len, replace = TRUE)
  #cat("Hello...")

  #Convert to single string
  if(single.element) {
    #cat("is it me you are looking for...")
   ans <-  paste( ans, collapse = "")
  }

  ##Format as FASTA with an ID line
  cat(paste(">len", len, "\n", sep=""))
  cat (ans)
  cat("\n")

  return(ans)
}
```

```
x <- generate_dna(20)
```

```
>len20
TATCCACCGAGGTTATGTGT
```

Q3. Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return "WQRTAG".

```
generate_protein <- function(len=9) {
  aa <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S","T"
  ans <- sample(x=aa, size=len, replace= TRUE)
  paste(ans, collapse="")
}
```

```
generate_protein(5)
```

```
[1] "MMILA"
```

Q4.Adapt and use your generate_protein() function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function for() or sapply() so that you do not have to call generate_protein() eight times by hand. Be sure to format your results in FASTA format ready to paste or upload as a query to NCBI BLASTp. For example:

```
for (l in 6:13) {
  cat(">",l, "\n", sep="")
 cat( generate_protein(l), "\n")
}
```

```
>6
CMAQEV
>7
WTRRVGP
>8
QRYSRSFS
>9
SPETKWLHS
>10
```

```
SHMLFGRLFG
>11
EWHGVRVLRNA
>12
CIQTNYMSRHEP
>13
GQISMDLHKPWEK
```

Q5 BLASTp search against nr — are your peptides "unique in nature"?

length	identity	cov	unique?
6	100%	100%	N
7	100%	100%	N
8	100%	100%	N
9	100%	89%	Y
10	75%	90%	Y
11	72.7%	100%	Y
12	90.91%	92%	Y
13	88.89%	69%	Y

5a. At which sequence length do your randomly generated peptides start to look "unique in nature" (i.e. no 100% coverage + 100% identity hit)? [1 pt]

at sequence length 9 they start to be unique in nature.

Q5b.Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20 for a peptide of length) and to the size of the known protein universe. [1 pt]

I believe that very short random peptides are found in the nr because total sequence space for peptides that are shorter are smaller thus statistically there are more combinations that can match to it when the sequence is shorter. Very short peptides (L) when put into the exponent of 20 has a lower and more limited possible sequences and are more likely to be a part of the protein universe. For example 20^6 = 6.4 x 10^7 and covers a greater fraction of known sequences whereas 20^13= 8.19 x10^16 is bigger than the known protein universe. Therefore less of the longer peptides are found in the nr.

Q6. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

The minimum length is around 9 amino acids of when a peptide becomes unique or longer where it tends to be unique and this is discussed in Q5b where 20^L present rapid expansion that when L increases (number of amino acids) it becomes less likely to fall between the known protein sequences on the database and thus be unique. Shorter peptides are thus more likely to be found within the known peptides dataset and this means that they cannot be distinguishable in biology. In immunology T-cells recognize peptides from MHC molecules that binds short peptides to allow for T helper cells distinguish if it is a self or nonself biological aspect and thus attack the pathgogen using the immune response. It would be a bad design if the immune system presents short peptides because then it would match more likely to self-proteins and this reduces specificity and increases the risk of T-cells not recognizing the pathogen thus the immune system does not attack it. This could make one become more sick if the immune system is not attacking the pathogen.